

September 24, 2024

1 Lâm Quang Phú

2 MSSV 21094601

Các thao tác như lọc ảnh (Image Filtering) thường được sử dụng như một bước tiền xử lý trong các ứng dụng Thị giác máy tính. Khái niệm về tần số tín hiệu, được biết đến từ xử lý tín hiệu số 1D, có thể được mở rộng đến tín hiệu trong không gian 2D, tức là hình ảnh. Thuật ngữ này chỉ tần số không gian trong tín hiệu 2D. Tần số không gian là thước đo cho biết mức độ thường xuyên mà các thành phần sóng hình sin (được xác định bằng biến đổi Fourier) của cấu trúc lặp lại trên mỗi đơn vị khoảng cách. Một vùng ảnh được coi là có thông tin tần số thấp nếu nó mượt mà và không có nhiều kết cấu. Một vùng ảnh được coi là có thông tin tần số cao nếu nó có nhiều kết cấu (cạnh, góc, v.v.). Trong Hình 1, các biến đổi Fourier tương ứng với hình ảnh logo bên trên chúng được hiển thị. Tại trung tâm của các biểu đồ phổ là các thành phần tần số bằng 0, trong khi di chuyển từ trung tâm ra bên ngoài, các thành phần tần số cao hơn được hiển thị với biên độ mã hóa bằng màu sắc. Các vùng màu xanh dương đậm lớn (tương ứng với các thành phần tần số có biên độ thấp) cho thấy hình ảnh đồng đều hơn, không có nhiều kết cấu nổi bật. Màu sắc ấm trong phổ chỉ ra biên độ cao hơn. Hình ảnh logo gốc chứa nhiều yếu tố hơn (đường viền, đối tượng, v.v.) nên phổ liên quan cũng có nhiều thành phần tần số cao hơn một cách đáng kể.

-
- Lọc thông thấp (Low Pass Filtering - LPF) về bản chất là thao tác làm mờ hoặc làm mịn một hình ảnh đầu vào. Việc làm mờ được thực hiện bằng cách loại bỏ các chi tiết tinh vi: nó cho phép thông tin tần số thấp đi qua và chặn thông tin tần số cao.
 - Lọc thông cao (High Pass Filtering - HPF) đại diện cho loại thao tác tăng cường cạnh hoặc làm sắc nét hình ảnh. Trong trường hợp này, thông tin tần số thấp bị loại bỏ và thông tin tần số cao được giữ lại.

2.1 2.1 Image filtering

Lọc ảnh (Image Filtering) là một thuật ngữ rộng được áp dụng cho nhiều kỹ thuật xử lý ảnh nhằm cải thiện một hình ảnh bằng cách loại bỏ các đặc điểm không mong muốn (ví dụ: nhiễu) hoặc cải thiện các đặc điểm mong muốn (ví dụ: độ tương phản tốt hơn). Các thao tác như làm mờ, phát hiện cạnh, làm sắc nét cạnh và loại bỏ nhiễu đều là các ví dụ của lọc ảnh.

Lọc ảnh là một thao tác cục bộ (local operation). Điều này có nghĩa là giá trị điểm ảnh tại vị trí (x, y) trong ảnh đầu ra phụ thuộc vào các điểm ảnh trong một vùng lân cận nhỏ xung quanh vị trí (x, y) trong ảnh đầu vào. Ví dụ, lọc ảnh sử dụng bộ lọc kích thước 3×3 (tương đương với phản hồi

xung trong không gian 2D) sẽ làm cho giá trị điểm ảnh tại vị trí (,) của ảnh đầu ra phụ thuộc vào các điểm ảnh đầu vào tại vị trí (,) và 8 điểm ảnh lân cận của nó, như hiển thị trong dưới đây.

Khi pixel đầu ra chỉ phụ thuộc vào sự kết hợp tuyến tính của các pixel đầu vào, chúng ta gọi bộ lọc là Bộ lọc tuyến tính. Ngược lại, nó được gọi là Bộ lọc phi tuyến.

2.1.1 2.1.1 Convolution

Bộ lọc tuyến tính được triển khai bằng cách sử dụng tích chập 2D (2D convolution). Các thao tác như làm mờ và phát hiện đường viền được thực hiện thông qua tích chập. Hãy xem xét một kernel (bộ lọc) kích thước 3x3:

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

chúng ta sẽ sử dụng nó để lọc điểm ảnh trung tâm màu vàng trong Hình 2. Giá trị điểm ảnh sau khi lọc được tính bằng cách nhân từng phần tử tương ứng của vùng màu xanh và vàng trong ảnh đầu vào với các phần tử của kernel tích chập, sau đó cộng tổng tất cả các tích lại:

$$\text{out}_{\text{center}} = 206 \times (-1) + 203 \times 0 + 205 \times 1 + 205 \times (-2) + 204 \times 0 + 205 \times 2 + 203 \times (-1) + 203 \times 0 + 205 \times 1 = 1$$

Để lọc toàn bộ hình ảnh, quy trình này được lặp lại cho từng điểm ảnh (kernel sẽ trượt qua hình ảnh từ trái sang phải, từ trên xuống dưới). Đối với hình ảnh màu, tích chập được thực hiện riêng biệt trên từng kênh màu. Tại các biên của ảnh, tích chập không được xác định rõ ràng. Có một số tùy chọn để xử lý:

- **Bỏ qua các điểm ảnh ở biên:** bằng cách bỏ qua các điểm ảnh ở biên, hình ảnh đầu ra sẽ nhỏ hơn một chút so với hình ảnh đầu vào.
- **Điền thêm số 0 (Zero padding):** hình ảnh đầu vào được điền thêm các số 0 ở các điểm ảnh biên để làm cho kích thước lớn hơn, sau đó mới thực hiện tích chập.
- **Sao chép biên (Replicate border):** một tùy chọn khác là sao chép các điểm ảnh biên của hình ảnh đầu vào và sau đó thực hiện phép tích chập trên hình ảnh lớn hơn này.
- **Phản chiếu biên (Reflect border):** tùy chọn được ưa thích là phản chiếu các điểm ảnh tại biên quanh đường viền. Phản chiếu đảm bảo sự chuyển tiếp cường độ điểm ảnh mượt mà tại vùng biên.

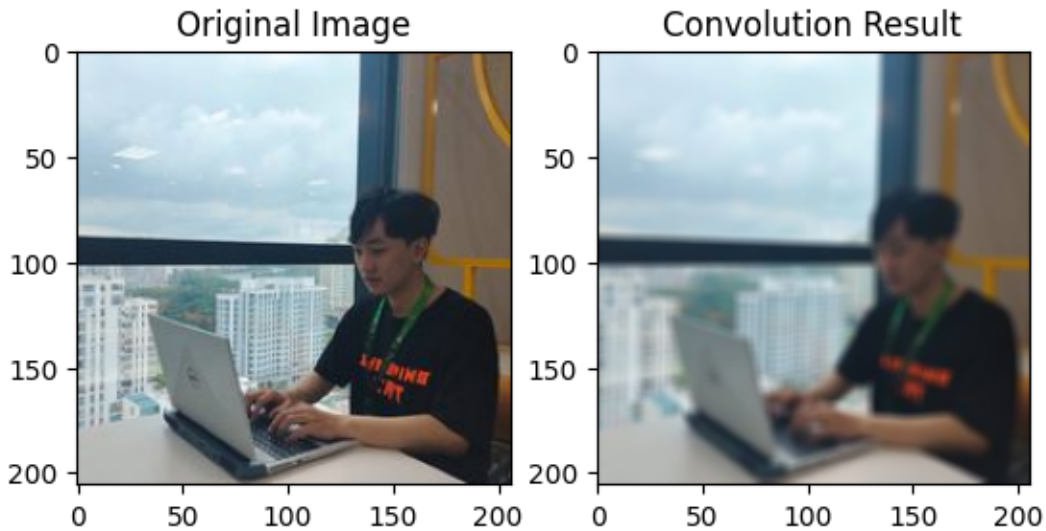
Trong OpenCV, phép tích chập được thực hiện bằng cách sử dụng hàm `filter2D`. Cú pháp của hàm như sau:

```
dst = cv2.filter2D(src, ddepth, kernel[, dst[, anchor[, delta[, borderType]]]])
```

```
[5]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
[6]: image = cv2.imread('test.jpg')
if image is None: # Check if file is not present
    print("Could not open the image")
ker_size = 5
# Box kernel: 5*5 kernel with the sum of all the elements equal to 1
kernel = np.ones((ker_size, ker_size), dtype=np.float32) / ker_size**2
print(kernel)
result = cv2.filter2D(image, -1, kernel, (-1, -1), delta=0, borderType =
cv2.BORDER_DEFAULT)
plt.figure()
plt.subplot(121);plt.imshow(image[...,:-1]);plt.title("Original Image")
plt.subplot(122);plt.imshow(result[...,:-1]);plt.title("Convolution Result")
plt.show()
```

```
[[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]
```

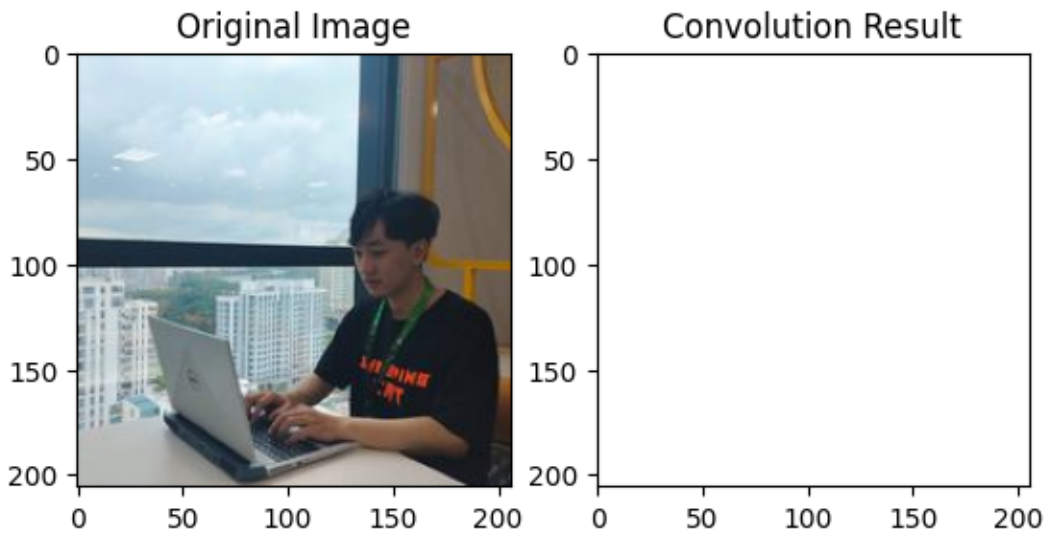


Tham số thứ hai của `cv2.filter2D` (depth) được đặt là -1, có nghĩa là độ sâu bit của ảnh đầu ra sẽ giống với ảnh đầu vào. Vì vậy, nếu ảnh đầu vào có kiểu dữ liệu là `uint8`, thì ảnh đầu ra cũng sẽ có cùng kiểu dữ liệu.

- **Ex. 2.1** Thay đổi kernel được sử dụng trong ví dụ trước và phân tích kết quả. Hãy thử sử dụng kernel mà không chuẩn hóa các giá trị theo số phần tử trong kernel. Điều này làm thay đổi đầu ra như thế nào mức độ sáng của hình ảnh so với hình ảnh đầu vào?

```
[7]: image = cv2.imread('test.jpg')
if image is None: # Check if file is not present
    print("Could not open the image")
ker_size = 5
# Box kernel: 5*5 kernel with the sum of all the elements equal to 1
kernel = np.ones((ker_size, ker_size), dtype=np.float32)
print(kernel)
result = cv2.filter2D(image, -1, kernel, (-1, -1), delta=0, borderType =
cv2.BORDER_DEFAULT)
plt.figure()
plt.subplot(121);plt.imshow(image[...,:-1]);plt.title("Original Image")
plt.subplot(122);plt.imshow(result[...,:-1]);plt.title("Convolution Result")
plt.show()
```

```
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
```



=> Thay đổi kernel bằng cách loại bỏ bước chuẩn hóa các giá trị theo số phần tử trong kernel. Điều này sẽ làm thay đổi độ sáng của hình ảnh đầu ra. Cụ thể, nếu không chia các phần tử trong kernel cho tổng số phần tử trong kernel, mỗi điểm ảnh trong kết quả sẽ trở thành tổng của các điểm ảnh trong vùng lân cận, thay vì trung bình của chúng, dẫn đến hình ảnh có thể bị quá sáng hoặc quá tối.

Ex. 2.2 Sửa đổi ví dụ trước bằng cách kiểm tra các kernel sau trên ảnh đầu vào “white_square.jpg” và “hop.jpg”

```

[11]: image1 = cv2.imread('white_square.png')
      image2 = cv2.imread('test.jpg')

      if image1 is None or image2 is None:
          print("Could not open one of the images")
      else:
          # Danh sách các kernel cần thử nghiệm
          kernels = {
              "h1": np.array([[0, 0, 0],
                              [0, 1, 0],
                              [0, 0, 0]], dtype=np.float32),

              "h2": np.array([[0, 0, 0],
                              [0, 0, 0],
                              [0, 0, 1]], dtype=np.float32),

              "h3": np.array([[ -1, -1, -1],
                              [-1,  8, -1],
                              [-1, -1, -1]], dtype=np.float32),

              "h4": (1/16) * np.array([[1, 2, 1],
                                       [2, 4, 2],
                                       [1, 2, 1]], dtype=np.float32)
          }

          # Áp dụng từng kernel lên mỗi ảnh và hiển thị kết quả
          for kernel_name, kernel in kernels.items():
              result1 = cv2.filter2D(image1, -1, kernel)
              result2 = cv2.filter2D(image2, -1, kernel)

              plt.figure(figsize=(10, 10))

              # Hiển thị kết quả trên ảnh "white_square.jpg"
              plt.subplot(2, 3, 1)
              plt.imshow(image1[..., ::-1])
              plt.title("Original: white_square")

              plt.subplot(2, 3, 2)
              plt.imshow(result1[..., ::-1])
              plt.title(f"Convolution with Kernel {kernel_name}")

              plt.subplot(2, 3, 3)
              plt.imshow(kernel, cmap='gray')
              plt.title("Kernel")

              # Hiển thị kết quả trên ảnh "hop.jpg"
              plt.subplot(2, 3, 4)

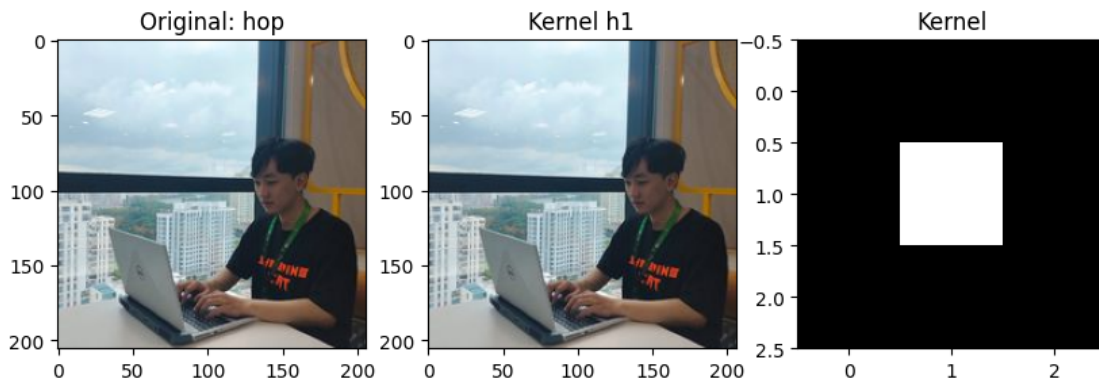
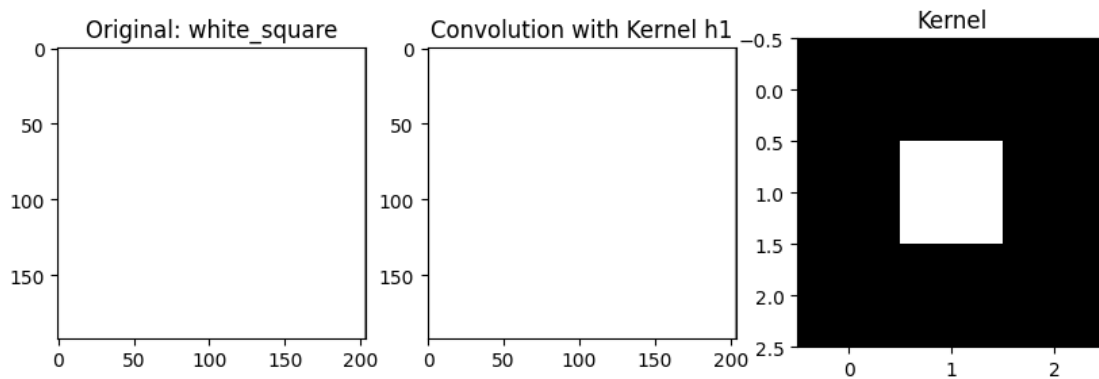
```

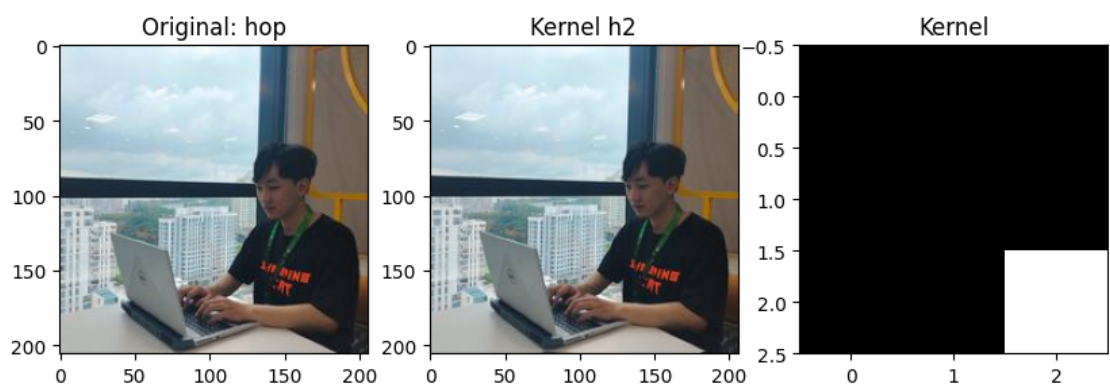
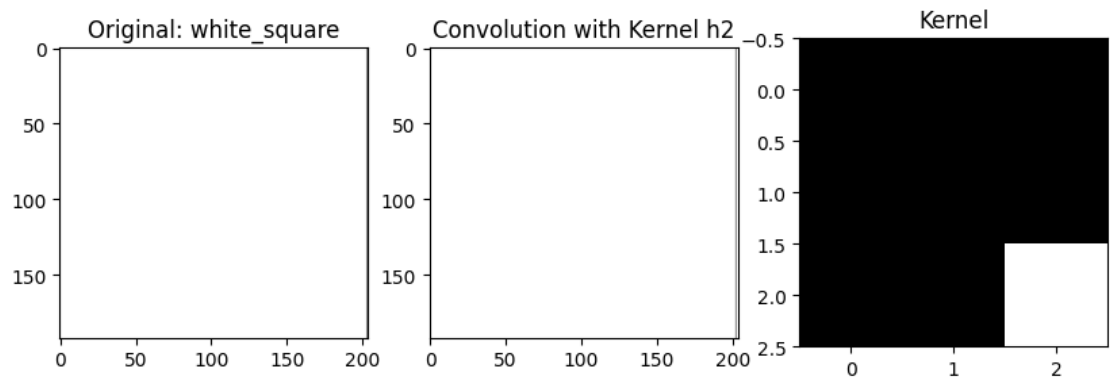
```
plt.imshow(image2[..., ::-1])
plt.title("Original: hop")

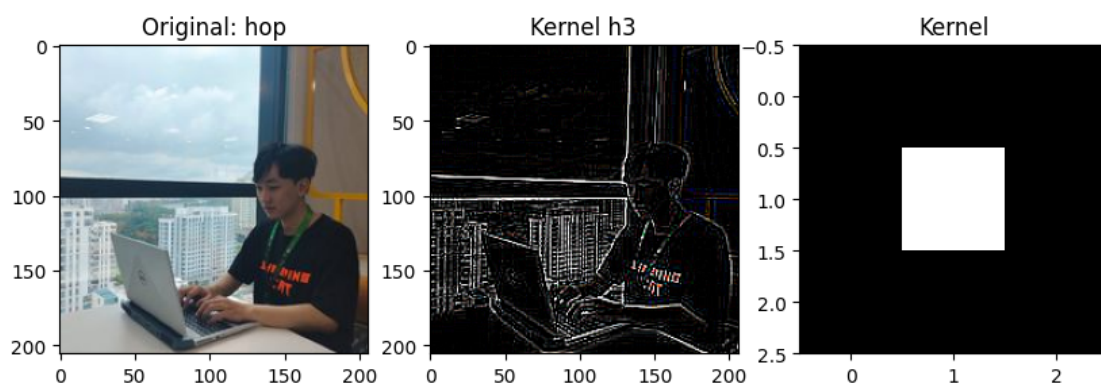
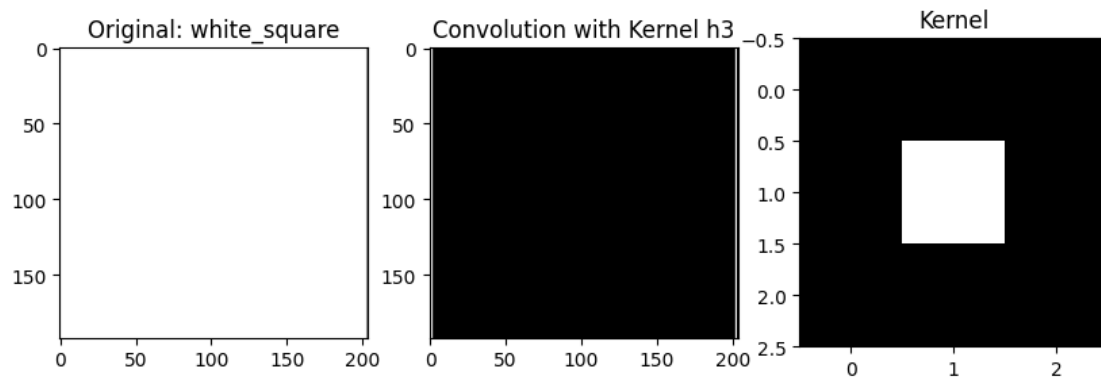
plt.subplot(2, 3, 5)
plt.imshow(result2[..., ::-1])
plt.title(f"Kernel {kernel_name}")

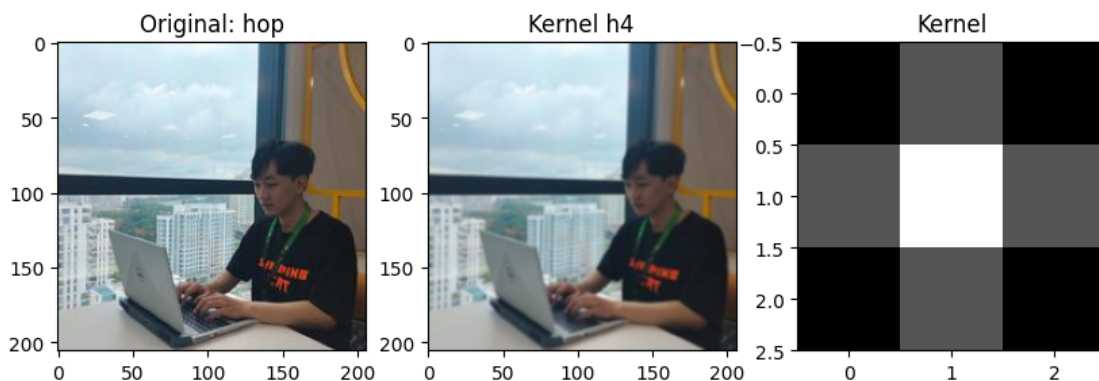
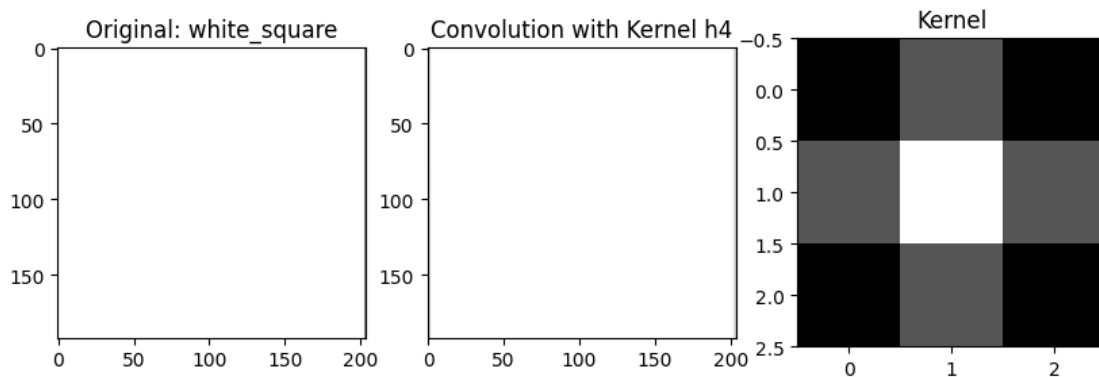
plt.subplot(2, 3, 6)
plt.imshow(kernel, cmap='gray')
plt.title("Kernel")

plt.show()
```









2.1.2 2.1.2 Box blur

Trong xử lý ảnh, việc làm mờ hoặc làm mịn ảnh đầu vào thường là cần thiết, và đây là một dạng của lọc thông thấp (LPF). Phương pháp làm mờ hộp (Box blur) là một kỹ thuật làm mịn giúp giảm các loại nhiễu khác nhau trong ảnh đầu vào. Kỹ thuật này có thể được triển khai theo 2 cách:

- **Cách 1:** Tạo một kernel (ví dụ như kernel kích thước 3x3)

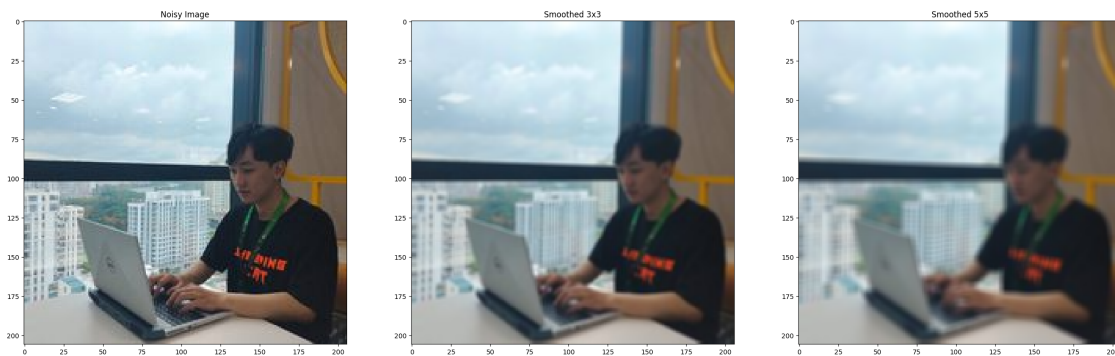
$$h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

và tích chập ảnh đầu vào với kernel này bằng cách sử dụng hàm `filter2D`.

- **Cách 2:** Sử dụng hàm `blur` trong OpenCV với cú pháp sau:

```
dst = cv2.blur(src, ksize[, dst[, anchor[, borderType]]])
```

```
[13]: img = cv2.imread('test.jpg')
# Apply box filter - kernel size 3
dst1 = cv2.blur(img,(3,3),(-1,-1))
# Apply box filter - kernel size 5
dst2 = cv2.blur(img,(5,5),(-1,-1))
plt.figure(figsize=[30,10])
plt.subplot(131);plt.imshow(img[...,:-1]);plt.title("Noisy Image")
plt.subplot(132);plt.imshow(dst1[...,:-1]);plt.title("Smoothed 3x3")
plt.subplot(133);plt.imshow(dst2[...,:-1]);plt.title("Smoothed 5x5")
plt.show()
```



Ex 2.3 Mô tả cả hai thao tác làm mịn theo kết quả cuối cùng: Mượt hơn bao nhiêu xuất hiện nền logo? Các đường viền thay đổi như thế nào khi kernel bị mờ?

1. Bộ lọc làm mịn với kernel (3 x 3)

- **Mô tả:**
 - Kernel (3 x 3) làm mịn hình ảnh bằng cách tính trung bình giá trị của các điểm ảnh trong vùng (3 x 3) xung quanh mỗi điểm ảnh.
 - Bộ lọc này giúp giảm nhiễu và làm mịn nhẹ nhàng các chi tiết nhỏ trong hình ảnh.
- **Kết quả:**
 - **Độ mượt:** Hình ảnh trông sẽ mượt hơn, giảm bớt nhiễu, nhưng vẫn giữ lại được các chi tiết tương đối rõ ràng.
 - **Nền:** Nền sẽ trở nên đều hơn, ít nhiễu hơn và các vùng có màu sắc tương đồng sẽ bị hòa trộn lại với nhau.
 - **Đường viền:** Các đường viền sẽ bị làm mềm, nhưng vẫn còn giữ được tương đối rõ ràng và sắc nét.

2. Bộ lọc làm mịn với kernel (5 x 5)

- **Mô tả:**
 - Kernel (5 x 5) có kích thước lớn hơn, làm mịn hình ảnh bằng cách tính trung bình giá trị của các điểm ảnh trong vùng (5 x 5) xung quanh mỗi điểm ảnh.
 - Bộ lọc này làm mờ mạnh hơn, giúp loại bỏ nhiễu nhiều hơn nhưng cũng làm mất đi nhiều chi tiết hơn.
- **Kết quả:**

- **Độ mượt:** Hình ảnh sẽ mượt hơn nhiều so với kernel (3 x 3). Nhiều sẽ giảm đáng kể, các chi tiết nhỏ và các vùng có màu sắc không đồng đều sẽ được làm mờ mạnh.
- **Nền:** Nền sẽ trông đồng nhất hơn, gần như không còn nhiễu hoặc các vùng lốm đốm không đều màu.
- **Đường viền:** Đường viền sẽ bị làm mờ nhiều hơn, các chi tiết nhỏ có thể bị mất, và các cạnh sắc nét sẽ bị làm mềm.

2.1.3 2.1.3 Gaussian filtering in OpenCV

Kernel hộp (Box kernel) được giải thích trước đó cân bằng đóng góp của tất cả các điểm ảnh trong vùng lân cận một cách đều nhau. Ngược lại, kernel làm mờ Gaussian (Gaussian Blur kernel) cân bằng đóng góp của các điểm ảnh lân cận dựa trên khoảng cách của điểm ảnh đó đến điểm ảnh trung tâm. Hình dạng của đường cong được điều khiển bởi một tham số duy nhất gọi là σ , tham số này điều chỉnh mức độ nhọn của đặc tính hình đôi:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Giá trị σ lớn hơn sẽ tạo ra một kernel làm mờ mạnh hơn. Một kernel Gaussian kích thước 5x5 với $\sigma = 1$ được cho bởi:

$$\frac{1}{337} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 20 & 33 & 20 & 4 \\ 7 & 33 & 55 & 33 & 7 \\ 4 & 20 & 33 & 20 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

Rõ ràng là điểm ảnh ở giữa có trọng số lớn nhất, trong khi các điểm ảnh càng xa thì trọng số càng nhỏ.

Một hình ảnh được làm mờ bằng kernel Gaussian sẽ trông ít bị mờ hơn so với khi sử dụng kernel hộp có cùng kích thước. Một lượng nhỏ làm mờ Gaussian thường được sử dụng để loại bỏ nhiễu khỏi hình ảnh. Nó cũng được áp dụng cho hình ảnh trước khi thực hiện các thao tác lọc ảnh nhạy cảm với nhiễu. Ví dụ, kernel Sobel được sử dụng để tính đạo hàm của ảnh là sự kết hợp giữa kernel Gaussian và kernel sai phân hữu hạn.

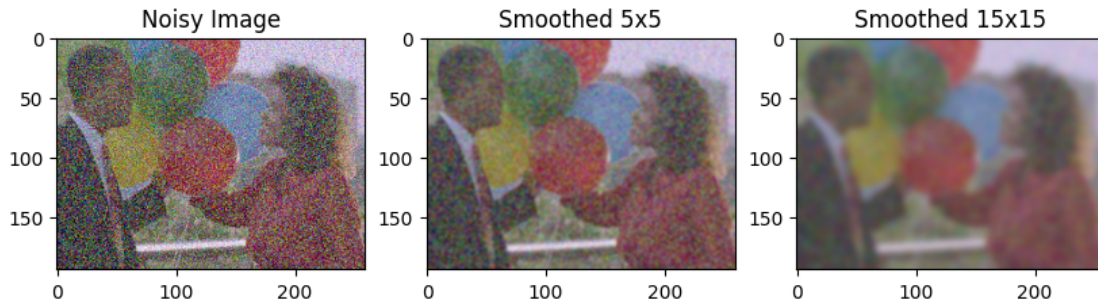
Hàm trong OpenCV thực hiện việc lọc Gaussian là `GaussianBlur` với cú pháp như sau:

```
dst = cv2.GaussianBlur(src, ksize, sigmaX[, dst[, sigmaY[, borderType]])
```

Ex. 2.4 Thêm các dòng mã còn thiếu, được biểu thị bằng nhận xét. Mô tả cả việc làm mịn hoạt động so sánh các hình ảnh cuối cùng: nền logo xuất hiện mượt mà hơn bao nhiêu? thể nào rồi đường viền trong hình ảnh thay đổi với hạt nhân mờ? Hãy thử lọc Gaussian cho hình ảnh thử nghiệm “logo_noise2.jpg” và “lina_noise.jpg” và xác định kích thước kernel tốt nhất.

```
[17]: img = cv2.imread('noise.jpg')
      # Apply gaussian blur with kernel 5x5 and sigmaX=0, sigmaY=0; output image dst1
      dst1 = cv2.GaussianBlur(img,(5,5),sigmaX = 0,sigmaY = 0)
```

```
# Apply gaussian blur with kernel 15x15 and sigmaX=3, sigmaY=3; ; output image
# dst2
dst2 = cv2.GaussianBlur(img,(15,15),sigmaX = 3,sigmaY = 3)
plt.figure(figsize=(10, 10))
plt.subplot(131);plt.imshow(img[...,:-1]);plt.title("Noisy Image")
plt.subplot(132);plt.imshow(dst1[...,:-1]);plt.title("Smoothed 5x5")
plt.subplot(133);plt.imshow(dst2[...,:-1]);plt.title("Smoothed 15x15")
plt.show()
```



2.1.4 Median filter

Lọc trung vị (Median blur filtering) là một kỹ thuật lọc phi tuyến, chủ yếu được sử dụng để loại bỏ nhiễu muối tiêu (salt-and-pepper noise) khỏi hình ảnh. Trong các ảnh màu, nhiễu muối tiêu có thể xuất hiện dưới dạng các chấm màu ngẫu nhiên nhỏ. Bộ lọc trung vị trong OpenCV nên có kernel hình vuông với chiều dài cạnh là một số lẻ. Bộ lọc làm mờ trung vị thay thế giá trị của điểm ảnh trung tâm bằng giá trị trung vị của tất cả các điểm ảnh trong vùng kernel. Giá trị trung vị được tính bằng cách sắp xếp các điểm ảnh theo thứ tự tăng dần và chọn giá trị ở giữa làm ước lượng.

Hãy xem xét một vùng 3x3 từ một hình ảnh bị nhiễu muối tiêu:

$$\begin{bmatrix} 60 & 62 & 59 \\ 61 & 255 & 65 \\ 65 & 60 & 63 \end{bmatrix}$$

Dễ dàng nhận thấy rằng giá trị tại điểm ảnh trung tâm cao hơn rất nhiều so với các điểm ảnh xung quanh, nên có khả năng bị ảnh hưởng bởi nhiễu. Nếu chúng ta sử dụng bộ lọc làm mờ hộp (Box Blur) để làm mịn nhiễu này, giá trị của điểm ảnh trung tâm sau khi lọc sẽ là:

$$\frac{60 + 62 + 59 + 61 + 255 + 65 + 65 + 60 + 63}{9} = 83.3$$

do đó vùng ảnh sau khi lọc hộp sẽ là:

$$\begin{bmatrix} 60 & 62 & 59 \\ 61 & 83 & 65 \\ 65 & 60 & 63 \end{bmatrix}$$

Kết quả đã được cải thiện rõ ràng, nhưng giá trị trung tâm vẫn cao hơn so với các điểm ảnh xung quanh. Sử dụng bộ lọc trung vị, các điểm ảnh được sắp xếp sẽ là:

[59, 60, 60, 61, **62**, 63, 65, 65, 255]

và giá trị ở giữa danh sách sắp xếp là 62. Vì vậy, sau khi lọc điểm ảnh trung tâm, vùng ảnh sẽ trở thành:

$$\begin{bmatrix} 60 & 62 & 59 \\ 61 & 62 & 65 \\ 65 & 60 & 63 \end{bmatrix}$$

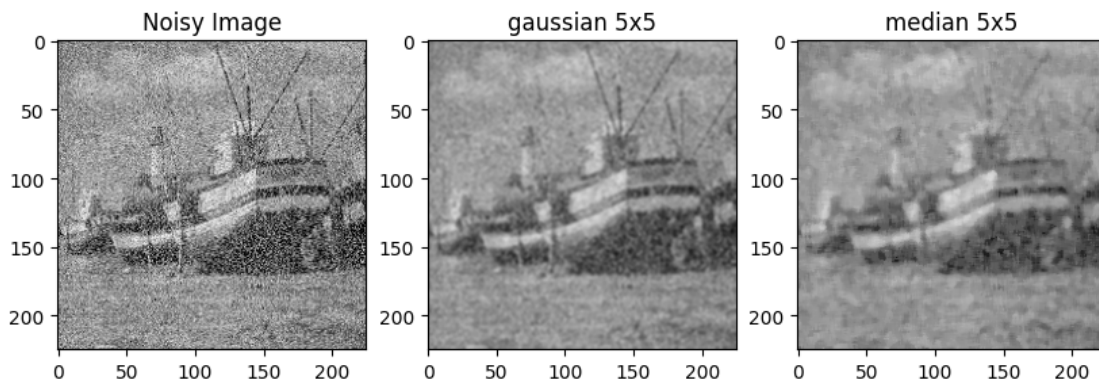
Quy trình này được lặp lại theo cùng một cách cho tất cả các điểm ảnh trong ảnh đầu vào.

• **Ex2.5** Thêm các dòng mã còn thiếu, được biểu thị bằng nhận xét. Loại kết cấu nào bị ảnh hưởng nhất bởi tiếng ồn muối tiêu? Bộ lọc làm mịn nào hoạt động tốt hơn trên các cạnh? theo thứ tự để bảo toàn các cạnh, nên sử dụng kernel lớn hơn hay kernel nhỏ hơn? Cái nào loại bỏ tiếng ồn nhất một cách hiệu quả? Hãy thử trải nghiệm với các kích thước hạt nhân khác!

```
[29]: img = cv2.imread('noise2.jpg')
# Apply gaussian blur with kernel 5x5 and sigmaX=0, sigmaY=0; output image dst1
kernelSize = 5
dst1 = cv2.GaussianBlur(img, (kernelSize, kernelSize), sigmaX = 0, sigmaY = 0)

# Performing Median Blurring with kernelSize=5 and store it in numpy array
# "dst2"
dst2 = cv2.medianBlur(img , kernelSize)

plt.figure(figsize = (10, 10))
plt.subplot(131);plt.imshow(img[...,:-1]);plt.title("Noisy Image")
plt.subplot(132);plt.imshow(dst1[...,:-1]);plt.title("gaussian 5x5")
plt.subplot(133);plt.imshow(dst2[...,:-1]);plt.title("median 5x5")
plt.show()
```



2.2 2.3 Image sharpenin

Làm sắc nét hình ảnh nhằm mục đích tăng cường các cạnh và làm nổi bật kết cấu nền. Ý tưởng này không phải mới, thực tế nó được bắt nguồn từ một kỹ thuật cổ điển được gọi là làm mờ không sắc nét (unsharp masking). Các bước sau đây là cần thiết để đạt được hiệu ứng làm mờ không sắc nét:

Bước 1: Làm mờ ảnh đầu vào để làm mịn kết cấu. Ảnh mờ chứa thông tin tần số thấp từ ảnh gốc. Giả sử I_b là ảnh đầu vào ban đầu và I là ảnh đã được làm mờ.

Bước 2: Trừ ảnh đã làm mờ khỏi ảnh gốc, $I - I_b$, để thu được thông tin tần số cao từ ảnh gốc.

Bước 3: Thêm lại thông tin tần số cao vào ảnh gốc và điều chỉnh lượng kết cấu tần số cao được thêm vào bằng cách sử dụng tham số α . Do đó, ảnh cuối cùng sau khi làm sắc nét là:

$$I_s = I + \alpha(I - I_b)$$

Hiện tại, việc làm sắc nét được thực hiện bằng cách sử dụng một kernel làm sắc nét đơn giản mô phỏng hành vi trên. Ảnh đầu vào được tích chập với kernel sau:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Kernel trên được thu được bằng cách sử dụng $\alpha = 1$ và ước lượng I_b bằng cách sử dụng kernel Laplacian:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Bước 1: Đọc ảnh đầu vào

```
[35]: image = cv2.imread('test.jpg')
      if image is None:
          print("Could not open the image")
```

Bước 2: Làm mờ ảnh đầu vào bằng bộ lọc Gaussian

```
[36]: blurred_image = cv2.GaussianBlur(image, (5, 5), 0) # Làm mờ ảnh với kernel 5x5
```

Bước 3: Tính thông tin tần số cao bằng cách trừ ảnh đã làm mờ khỏi ảnh gốc

```
[37]: high_freq_details = image - blurred_image
```

Bước 4: Điều chỉnh lượng thông tin tần số cao và thêm vào ảnh gốc


```
[44]: alpha = 1.5 # Hệ số điều chỉnh
sharpened_image = cv2.addWeighted(image, 1.0, high_freq_details, alpha, 0)

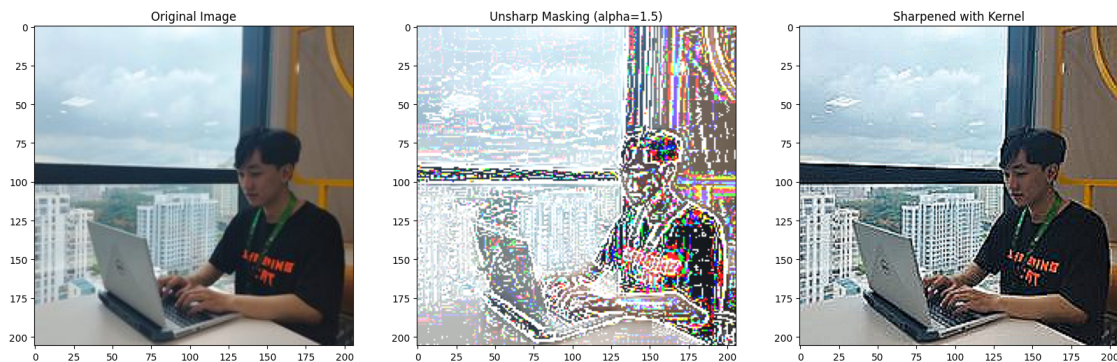
# Hoặc sử dụng kernel làm sắc nét để làm sắc nét trực tiếp
# Kernel làm sắc nét với alpha = 1
sharpening_kernel = np.array([[ 0, -1,  0],
                              [-1,  5, -1],
                              [ 0, -1,  0]], dtype=np.float32)

# Áp dụng kernel làm sắc nét lên ảnh gốc
sharpened_image_with_kernel = cv2.filter2D(image, -1, sharpening_kernel)

# Hiển thị kết quả
plt.figure(figsize=(20, 10))

plt.subplot(131); plt.imshow(image[..., ::-1]); plt.title("Original Image")
plt.subplot(132); plt.imshow(sharpened_image[..., ::-1]); plt.title("Unsharp_
↳Masking (alpha=1.5)")
plt.subplot(133); plt.imshow(sharpened_image_with_kernel[..., ::-1]); plt.
↳title("Sharpened with Kernel")

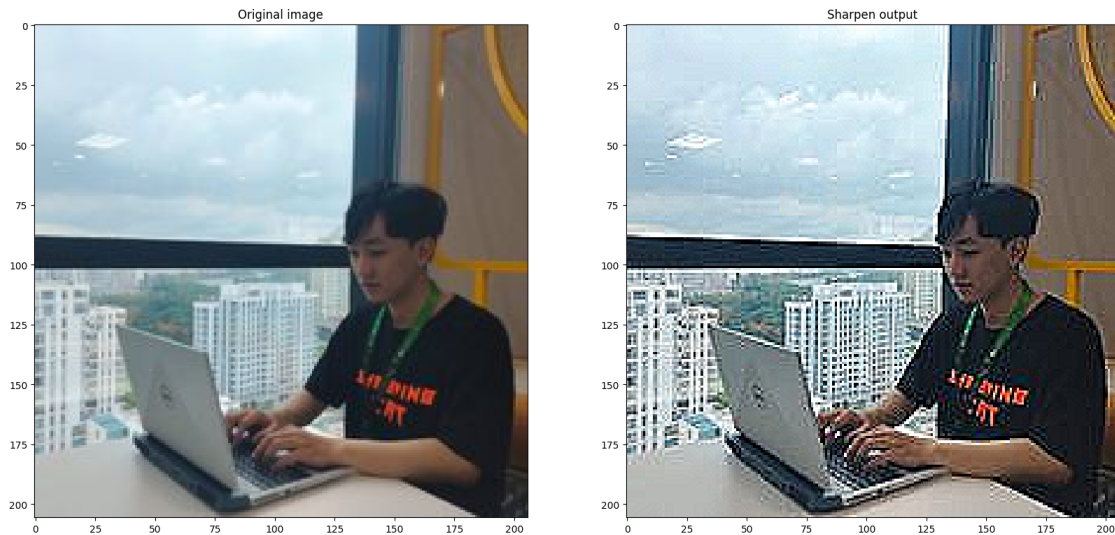
plt.show()
```



Sử dụng thư viện

```
[30]: img = cv2.imread("test.jpg")
# Sharpen kernel
sharpen = np.array([ 0, -1,  0],
                  [-1,  5, -1],
                  [ 0, -1,  0]), dtype="int")
sharpImg = cv2.filter2D(img, -1, sharpen)
plt.figure(figsize=[20,10])
plt.subplot(121);plt.imshow(img[...,::-1]);plt.title("Original image")
plt.subplot(122);plt.imshow(sharpImg[...,::-1]);plt.title("Sharpen output")
```

```
plt.show()
```



- 2.9 Các thành phần kết cấu được làm nổi bật bởi hiệu ứng làm sắc nét là gì? Thử nghiệm cũng với các hình ảnh khác (parrots.jpg). Mô tả loại kết cấu nổi bật hơn sau khi làm sắc nét lọc.

```
[50]: # Đọc ảnh đầu vào
image = cv2.imread('parrots.jpg', 0)
if image is None:
    print("Could not open the image 'parrots.jpg'")
else:
    # Bước 1: Làm mờ ảnh đầu vào bằng bộ lọc Gaussian
    blurred_image = cv2.GaussianBlur(image, (5, 5), 0)

    # Bước 2: Tính thông tin tần số cao bằng cách trừ ảnh đã làm mờ khỏi ảnh gốc
    high_freq_details = image - blurred_image

    # Bước 3: Điều chỉnh lượng thông tin tần số cao và thêm vào ảnh gốc
    alpha = 1.5 # Hệ số điều chỉnh
    sharpened_image = cv2.addWeighted(image, 1.0, high_freq_details, alpha, 0)

    # Kernel làm sắc nét
    sharpening_kernel = np.array([[ 0, -1,  0],
                                   [-1,  5, -1],
                                   [ 0, -1,  0]], dtype=np.float32)

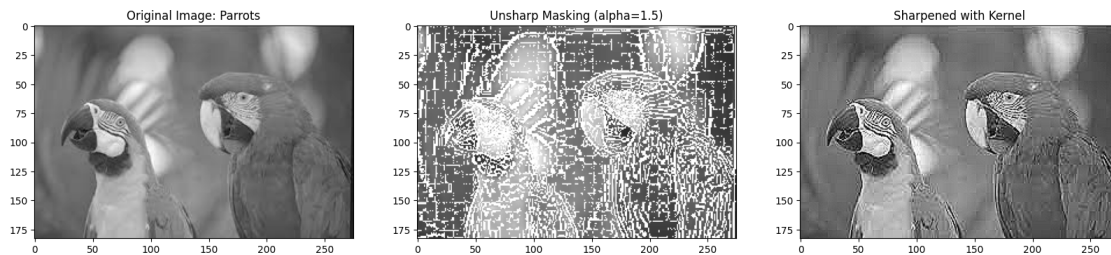
    # Áp dụng kernel làm sắc nét lên ảnh gốc
    sharpened_image_with_kernel = cv2.filter2D(image, -1, sharpening_kernel)
```



```
# Hiển thị kết quả
plt.figure(figsize=(20, 10))

plt.subplot(131); plt.imshow(image,cmap='gray'); plt.title("Original Image: Parrots")
plt.subplot(132); plt.imshow(sharpened_image,cmap='gray'); plt.title("Unsharp Masking (alpha=1.5)")
plt.subplot(133); plt.imshow(sharpened_image_with_kernel,cmap='gray'); plt.title("Sharpened with Kernel")

plt.show()
```



Hiệu ứng làm sắc nét ảnh (sharpening) có mục đích chính là làm nổi bật các cạnh và kết cấu trong hình ảnh. Khi áp dụng lọc làm sắc nét lên hình ảnh, các thành phần kết cấu nổi bật hơn là:

1. Các cạnh (Edges):

- Những đường viền giữa các vùng có độ tương phản cao, chẳng hạn như đường viền giữa các vật thể, các đường nét rõ ràng, sẽ được làm nổi bật rõ ràng hơn. Điều này làm cho các đối tượng trong ảnh trông sắc nét và rõ ràng hơn.

2. Kết cấu chi tiết (Fine Textures):

- Những kết cấu nhỏ như lông vũ của con vẹt, vảy cá, hoặc những chi tiết nhỏ trên bề mặt của vật thể cũng sẽ được làm nổi bật hơn. Hiệu ứng làm sắc nét sẽ giúp nhấn mạnh các chi tiết này, khiến chúng trở nên rõ ràng và dễ nhận biết hơn.

3. Độ chi tiết tổng thể (Overall Detail):

- Toàn bộ chi tiết nhỏ trong ảnh sẽ trở nên rõ ràng hơn, tạo cảm giác ảnh trở nên sắc nét và rõ nét hơn.

-
- Kỹ thuật làm sắc nét giúp làm nổi bật các chi tiết trong ảnh, đặc biệt là các đường viền và kết cấu nhỏ.
 - Tuy nhiên, cần chú ý không làm sắc nét quá mức vì có thể gây ra hiệu ứng halo hoặc làm mất đi tính tự nhiên của ảnh. Việc điều chỉnh hệ số (α) một cách hợp lý là cần thiết để đạt được kết quả tốt nhất.

[]: